

MLE Deep Tech (Generalist): ML Project Review

Background

At Riot, internal teams produce a lot of documentation — post-mortems, design specs, onboarding guides, runbooks, and knowledge base articles. Over time, the internal wiki has grown to tens of thousands of pages. Engineers frequently report that they can't find what they need, and new hires spend significant time searching for relevant docs during onboarding.

The ML engineering team has been asked to build a recommendation system that suggests relevant articles to employees based on what they're currently reading. A junior engineer on the team has completed a proof-of-concept and written up their approach in the following project report. Your team lead has asked you to review the report and come prepared to discuss it.

Task

The following is a **project report** describing how the recommendation system was built and evaluated. It has strengths and weaknesses — some things are done well, some are done poorly, and some important things are missing entirely.

Your task is to read it carefully and come to the interview ready to discuss:

- What was done well
- What you would change or fix, and why
- What's missing that should be there
- Questions you would ask the author

You **are** allowed to use any tooling you want to help you prepare, but we ask you **do not use any supplemental tools during the interview**. We want to know what **you** think.

Format: The interview is a working session, not an oral exam. Think of it as reviewing a colleague's work and discussing what you'd change. The interviewer will ask follow-up questions along the way, the same way they would in a real team discussion. Come ready to screen share — you may be asked to sketch or diagram your ideas. The interview is 1 hour, but after intros and your questions at the end, we'll have about 45 minutes of dedicated discussion time.

NB: You are recommended to take less than 2 hours preparing for this. Focus on what you think matters most.

Project Report: Wiki Article Recommender

1. Problem Statement

Build a system that, given the article a user is currently viewing, recommends 5 other articles they might find useful.

2. Dataset

We pulled all wiki articles from the last 3 years. After filtering out stubs (< 50 words), we have **42,000 articles**. We also have a **click log** of which articles users navigated to from other articles (i.e., if a user was reading article A and then clicked through to article B, that's a positive signal).

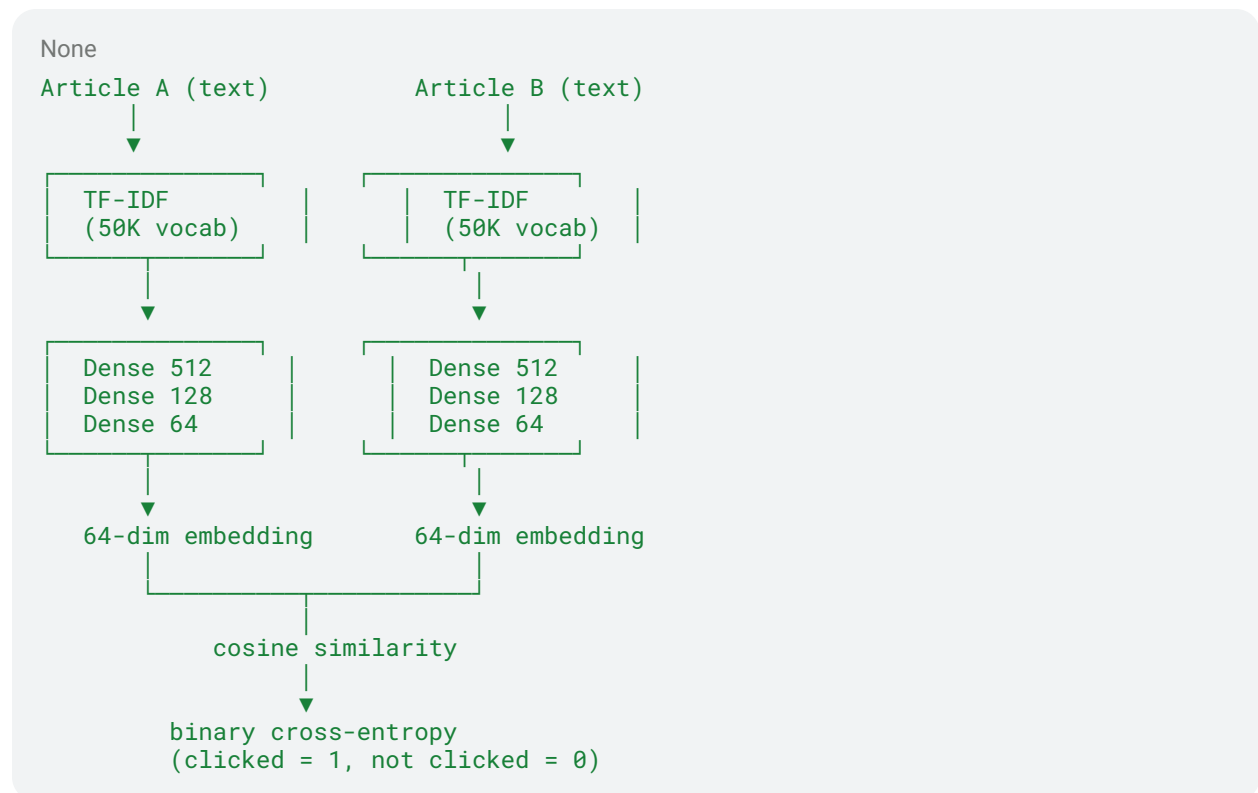
Summary statistics:

- 42,000 articles
- 1.2M click-through events over 12 months
- Average article length: ~800 words
- 3,400 unique users in the click log

We split the data 80/20 random for train and test.

3. Approach

We use a two-tower model:



Both towers share weights. Input is TF-IDF vectors. The model learns 64-dimensional article embeddings. We train on (article A, article B, clicked) triples.

Negative sampling: For every positive click pair, we sample 4 random articles as negatives.

Training details:

- Framework: PyTorch
- Optimizer: Adam, lr=0.001
- Batch size: 1024
- Epochs: 50
- Training time: ~2 hours on a single V100

4. Results

Metric	Train	Test
AUC	0.97	0.88
Accuracy	0.93	0.85
Precision@5	—	0.31

We're happy with the AUC and accuracy numbers. Precision@5 is lower, but recommendations are subjective so this is expected.

5. Serving Plan

Once the model is trained, we precompute embeddings for all 42K articles and store them in a vector database. At query time, we look up the current article's embedding and return the 5 nearest neighbors by cosine similarity.

We plan to update the embeddings weekly by retraining on the latest click data and recomputing all embeddings.

6. Next Steps

- Deploy behind an internal API
- Add a feedback button ("was this helpful?") to collect explicit signal
- Explore using BERT embeddings instead of TF-IDF

Thank you for your time! We look forward to chatting

with you